Seg2Vector: Remote Sensing Road Graph Extraction via Segmentation-to-Vector Transformation

Shengqi Wang, Dongyang Zhang, Junmin Liu, *Member, IEEE*, Xiangyong Cao, and Chengli Tan

***Abstract*—Road graph extraction from high-resolution remote sensing images is a pivotal task for applications such as urban planning and autonomous driving. However, segmentation-based approaches often suffer from a complex entanglement between semantic feature learning and topological structure modeling. This coupling can result in a misalignment between training objectives and the requirements of inference. To address these limitations, we propose Seg2Vector, a novel framework designed to directly transform segmentation masks into road graphs. Instead of depending on internal deep features extracted from the segmentation model, our method introduces a dedicated vector model that learns geometric representations directly from the segmentation masks. This design effectively decouples the topological reasoning task from the segmentation process. Moreover, unlike prior methods that explicitly predict connectivity between vertices—which can be ambiguous and challenging—the vector model predicts directional vectors for each vertex. These vectors are then processed by a heuristic algorithm to construct the final road graph. Seg2Vector allows the segmentation model and the vector model to be trained independently, enabling a plug-and-play capability across diverse datasets. Extensive experiments on three public benchmarks validate the effectiveness of our method, demonstrating superior performance, particularly in terms of topological correctness. The code is available at https://github.com/ShqWW/seg2vector**

***Index Terms*—Remote sensing, road graph extraction, semantic segmentation, vector representation.**

I. Introduction

ROAD graph extraction [1] from remote sensing images is a critical task in autonomous driving [2], navigation systems [3], and smart city construction [4]. Unlike road segmentation, which focuses solely on classifying road pixels, road graph extraction [5] is a significantly more complex challenge. This task requires the model to not only accurately identify the geometric coordinates of road vertices, but also precisely infer the topological connectivity between these vertices to form a coherent graph structure.

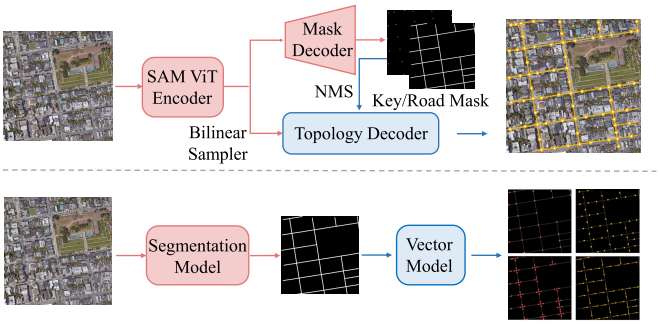


Fig. 1. Pipelines of SAM-Road [13] and our proposed Seg2Vector. The first row illustrates the SAM-Road pipeline, which extracts vertices from the segmentation mask and uses a learned topology decoder to predict connectivity between vertices. The second row depicts the proposed Seg2Vector pipeline, which decouples the segmentation task from graph extraction and predicts vertices with directional vectors instead of connectivity.

Earlier methods for road graph extraction can be broadly divided into two categories: iterative methods [6], [7], [8], [9] and global methods [10], [11], [12]. Iterative methods generate graphs by iteratively growing from existing vertices, simultaneously creating new vertices and connecting them with new edges. However, these algorithms are computationally expensive when processing high-resolution images. On the contrary, global methods predict all vertices together with their geometric attributes and subsequently infer their topological relationships. However, due to the complexity of these topological associations, constructing them accurately is often difficult, which frequently leads to suboptimal performance.

Recently, large-scale vision foundation models have developed rapidly, exemplified by Vision Transformers [14] and SAM [15]. In parallel, the remote sensing field has been actively working on foundation models specialized for this domain, such as Skysense [16] and HyperSIGMA [17]. Some studies have attempted to leverage these models as pretrained backbones for road graph extraction. Representative works include SAM-Road [13] and SAM-Road++ [18], which leverage the strong segmentation capability of SAM to generate high-quality road masks. Subsequently, vertices are extracted from these masks, and a learnable module is employed to predict the topological relationships between the vertices, as illustrated in the top row of Fig. 1.

Received 2 April 2026; revised 12 July 2026; accepted 15 August 2026. Date of publication 20 August 2026; date of current version 28 August 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62276208, Grant 12501710, and Grant 62272375; and in part by the Natural Science Basic Research Program of Shaanxi Province under Grant 2024JC-JCQN-02. *(Corresponding authors: Junmin Liu; Chengli Tan.)*

Shengqi Wang and Dongyang Zhang are with the School of Mathematics and Statistics, Xi'an Jiaotong University, Xi'an 710049, China.

Junmin Liu is with the School of Mathematics and Statistics, Xi'an Jiaotong University, Xi'an 710049, China, and also with the SGIT AI Laboratory, State Grid Corporation of China, Xi'an 712000, China (e-mail: junminliu@mail.xjtu.edu.cn).

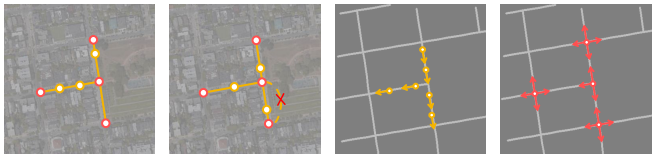
Xiangyong Cao is with the School of Computer Science and Technology and the Ministry of Education Key Laboratory for Intelligent Networks and Network Security, Xi'an Jiaotong University, Xi'an 710049, China (e-mail: caoxiangyong@mail.xjtu.edu.cn).

Chengli Tan is with the School of Mathematics and Statistics, Northwestern Polytechnical University, Xi'an 710129, China (e-mail: cltan023@nwpu.edu.cn).

Digital Object Identifier 10.1109/TGRS.2026.3725197

1558-0644 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c) (d)

Fig. 2. Comparison of traditional graph representations and the proposed directional vector approach. (a) and (b) Two graph representations of the same road topology. Red vertices: fixed intersections; yellow: variable points along roads. Hence, graph representations are nonunique—an edge's presence [e.g., dashed edge removed in (b)] depends on neighboring vertices as well as endpoints. (c) Bidirectional road vectors (yellow). For a standard road vertex, two opposite directional vectors are derived (one shown). (d) Multidirectional key vectors. At a key vertex where roads meet, multiple directional vectors are generated.

Although these pretrained models achieve impressive performance in road graph extraction, several limitations remain. First, training the connectivity prediction module (e.g., the topology decoder) is challenging due to the inherent properties of graphs. For instance, a single road instance can be represented by diverse graph structures, complicating the training of the connectivity prediction module, as illustrated in Fig. 2. The two red vertices in Fig. 2(a) are connected, whereas in Fig. 2(b), they are not connected due to the presence of an extra yellow vertex. This implies that the topological connectivity between two vertices is not determined solely by the candidate vertices themselves but is also influenced by the surrounding sampled vertices. SAM-Road and SAM-Road++ address this issue by employing a Breadth-First Search (BFS) algorithm to adapt the sampled vertices and the ground-truth graph at each training step to calculate the loss for the topology decoder. Therefore, training such a topology decoder is computationally expensive and time-consuming.

Nevertheless, the topology decoder lacks interpretability, making it difficult to discern how the module extracts the semantic context of the road and infers topological connectivity between vertices. Furthermore, SAM-Road [13] often exhibits suboptimal training performance. Because the topology decoder depends on well-predicted masks that are not yet available in the early stages of training, a dependence bottleneck arises. SAM-Road addresses this issue by employing a teacher-forcing strategy that directly samples vertices from the ground-truth graphs. However, this approach results in a significant discrepancy between training and inference. SAM-Road++ [18] attempts to mitigate this gap by resampling from the predicted masks during training; nevertheless, this operation is computationally expensive, and a marked gap between training and inference persists.

While SAM-based models [13], [18] struggle to capture vertex connectivity and their topology decoders are time-consuming to train, they nonetheless offer significant advantages for road segmentation. Notably, humans can easily infer topological connectivity directly from high-quality segmentation results without referring to the original images. This observation prompts two critical questions.

1) Is it feasible to derive the road structure representations directly from segmentation probability maps, thereby eliminating the need for deep internal features extracted from the original image?
2) How can the geometric structure of roads be represented in an efficient, learnable way, rather than by directly generating the vertex connectivity?

The first question is about the decoupling of semantic segmentation and road structure modeling, and the second question concerns the design of a more explicit and learnable representation for the road network structure. To address these two questions, we propose Seg2Vector, a novel decoupled and vectorized framework. The overall pipeline of our method is illustrated in the second row of Fig. 1.

Unlike SAM-Road [13] and SAM-Road++ [18], which rely on the internal deep features of the backbone to learn vertex connectivity, we decouple the segmentation and graph extraction tasks using two separate models: a segmentation model and a vector model. The two models can be trained independently and function as distinct modules during the inference stage. In particular, the vector model receives only the road mask as input to predict representations for the road structure. Thus, the first question is addressed.

Rather than directly inferring vertex connectivity, we propose a paradigm that predicts directional vectors, as illustrated in Fig. 2(c) and (d). Unlike vertex connectivity, which relies on multipoint relationships, directional vectors operate at a pointwise level. This approach renders the representation more explicit and easier to learn, as it directly encodes the local geometric structure. Subsequently, a heuristic algorithm named *Vector-to-Graph* (V2G) is employed to construct the final road graph. Thus, the second question is addressed.

While similar strategies have been explored in prior studies [11], [12], [19], those approaches typically depend on a backbone's deep internal features for vector learning, resulting in coarse-grained representations. In particular, these methods often account for only simple bidirectional road vertices, thereby overlooking complex intersections with multiple directions. On the contrary, we introduce a fine-grained vector representation model that incorporates both bidirectional road vectors and multidirectional key vectors, as shown in Fig. 2(c) and (d). This rich semantic information facilitates significantly more accurate graph extraction.

1) We propose Seg2Vector, a novel framework that reformulates graph extraction as directional vector learning. We introduce a decoupled paradigm that separates the segmentation and graph extraction tasks, allowing for independent training and inference. This design enhances flexibility and enables plug-and-play integration across different model architectures and datasets without performance degradation.
2) We introduce a novel representation paradigm utilizing directional vectors. Compared to previous vector-based methods, our method captures the local geometric structure of roads and intersections more explicitly. We further propose a heuristic algorithm, V2G, to transform vector predictions into the final graph, achieving superior accuracy and efficiency.
3) Extensive evaluations are performed on three public benchmarks. The results show that our method

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.

outperforms existing methods on the road graph extraction task.

II. Related Work

Road extraction [20], [21], [22] refers to the automatic identification and delineation of road networks within remote sensing images. This field can be broadly divided into two primary tasks: road segmentation and road graph extraction.

A. Road Segmentation

Road segmentation focuses on the pixelwise classification of images to distinguish between road and nonroad areas. To support this task, numerous datasets [23], [24] and diverse methodologies [25], [26], [27], [28], [29] have been proposed. To facilitate further research, several datasets featuring graph annotations have been introduced [10], [18], [30]. However, segmentation results are often insufficient for applications such as navigation and urban planning, which require a structured representation of the road network. Road graph extraction addresses this limitation by not only identifying road positions but also inferring the topological structure of the network, including vertices and edges. In this work, we focus exclusively on the road graph extraction task, although our method utilizes segmentation results as an intermediate step.

B. Road Graph Extraction

Existing approaches to road graph extraction can be broadly categorized into iterative methods and global methods. Iterative methods [6], [7], [8], [9] generate graphs by progressively expanding from existing vertices, simultaneously creating new vertices and connecting them with edges. RoadTracer [31], a pioneering method in this paradigm, predicts subsequent vertices from current ones and expands the graph iteratively. VecRoad [7] introduces a flexible step-size detection technique capable of identifying nontrivial points. RNGDet [8] and its successor, RNGDet++ [9], significantly improve the performance of iterative methods by combining CNNs with Transformers. However, due to their reliance on serial execution, these methods are computationally intensive when processing high-resolution images. Furthermore, errors tend to accumulate during the stepwise growth process.

On the contrary, global methods generate all vertices simultaneously and subsequently infer their connectivity using either learnable or heuristic mechanisms. For example, Sat2Graph [10] employs CNNs with downsampling layers to detect vertices with associated direction angles and leverages handcrafted criteria to connect them. Similarly, TopoRoad [12] and PaLiS [11] detect vertices along with corresponding bidirectional road vectors, connecting them via heuristic criteria. However, these approaches typically consider only simple bidirectional road vertices, thereby neglecting intersections involving multiple directions. GLD-Road [32] incorporates a directional descriptor to characterize the directional features of road nodes. SAM-Road [13] leverages the powerful segmentation capabilities of SAM to generate high-quality road masks, extracts vertices from these masks, and employs a learnable module to predict connectivity. However, training the connectivity module requires ground-truth labels. To address this, SAM-Road utilizes a teacher-forcing strategy to generate pseudolabels, but this introduces a discrepancy between training and inference. Similarly, kLCRNet [33] uses a lightweight backbone for the whole pipeline, thereby achieving faster inference, but the same issue persists. SAM-Road++ [18] attempts to mitigate this gap by resampling from the predicted masks during training; nevertheless, this operation remains computationally expensive. Compared with SAM-Road and SAM-Road++, our approach decouples the segmentation and graph extraction tasks while preserving the segmentation capability of SAM. By learning directional vectors instead of directly inferring connectivity, we offer a solution that is both more explicit and easier to learn.

C. Vector Representation

TopoRoad [12] and PaLiS [11] utilize directional vectors for road graph extraction. Similar vector representations have been extensively explored across various computer vision domains. For instance, oriented object detection requires the prediction of bounding box orientation; works such as PSC [34] and DCL [35] have specifically addressed the angle encoding challenges in this context. Moreover, other works like [36] and [37] employ vector sequences to delineate object boundaries for instance segmentation, while line segment detection methods [38], [39] also rely on directional vector prediction. Collectively, these studies provide valuable insights into vector representation design and training strategies. In Section III, we investigate these issues specifically within the context of road graph extraction, focusing on the design of bidirectional road vectors and multidirectional key vectors.

III. Method

The road graph extraction task can be formulated as follows: given a satellite image $\boldsymbol{I} \in \mathbb{R}^{3\times H\times W}$ of size $H \times W$, the goal is to extract a road graph $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ and $\mathcal{E}$ represent the sets of vertices and edges. The overall architecture of the proposed Seg2Vector is illustrated in Fig. 3. The framework consists of three main components: the segmentation model, the vector model, and the V2G module. We will introduce these components in Sections III-A, III-B, and III-D.

A. Segmentation Model

For the input image $\boldsymbol{I}$, the segmentation model aims to generate an accurate road mask $\boldsymbol{S}_r = (y_{ij})_{H\times W}$. Similar to SAM-Road [13] and SAM-Road++ [18], we adopt the pretrained encoders of SAM and SAM2 as the backbones of the segmentation model and fine-tune them for our task. In particular, the SAM encoder is equipped with a new segmentation head, while SAM2 is enhanced with an additional *Feature Pyramid Network* (FPN) to accommodate its multi-scale architecture. Unlike SAM-Road and SAM-Road++, which predict both the road mask and the key mask, the segmentation model in our Seg2Vector focuses solely on the road mask $\boldsymbol{S}_r$, while the key mask is generated by the vector model. This design choice is motivated by the observation that the key mask represents a subset of the road mask $\boldsymbol{S}_r$. Given the ground

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.

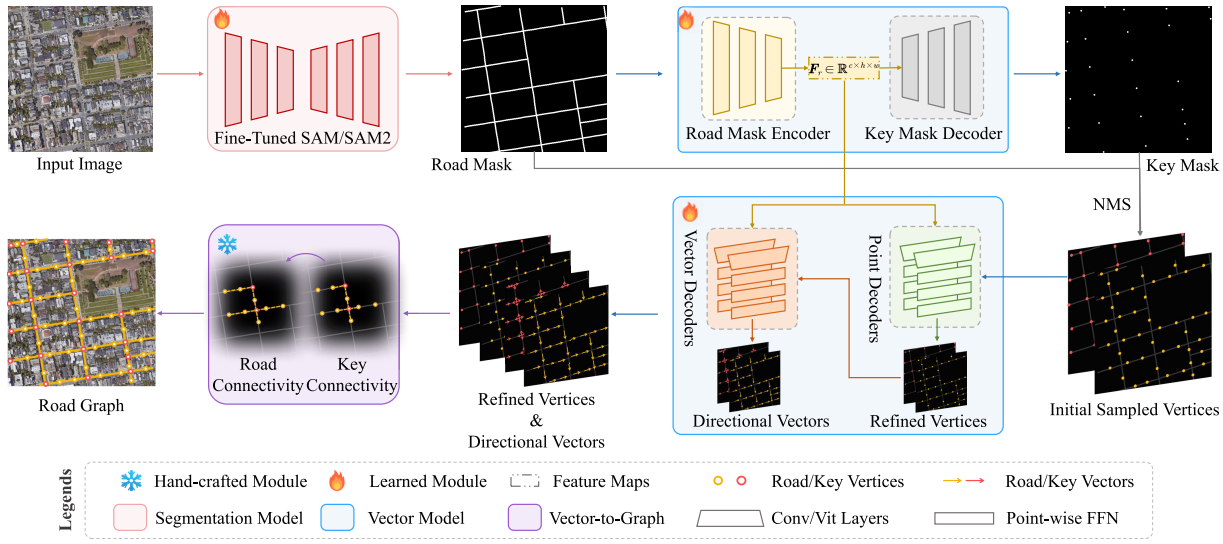


Fig. 3. Overall architecture of Seg2Vector consists of three main components: segmentation model (red), vector model (blue), and V2G module (purple). The segmentation model produces road masks, which feed into the vector model to output vertices with directional vectors. The V2G module then constructs a road network graph from these vertices. Decoupling the segmentation and vector models enables independent training and plug-and-play integration.

truth mask $\hat{y}_{ij}$ of road mask y_{ij}, the road segmentation loss $\mathcal{L}_s^r$ is defined as follows:

$$\mathcal{L}_s^r = \mathcal{L}_{BCE} + w_T \mathcal{L}_T \tag{1}$$

where w_T denotes the weight factor. The first term $\mathcal{L}_{BCE}$ in (1) represents the well-known pixel wise binary cross entropy loss

$$\mathcal{L}_{BCE} = -\sum_{i,j} \left[\hat{y}_{ij} \log\left(y_{ij}\right) + \left(1-\hat{y}_{ij}\right) \log\left(1-y_{ij}\right)\right]. \tag{2}$$

The second term $\mathcal{L}_T$ in (1) is the Tversky loss [40], which is defined as

$$\mathcal{L}_T = 1 - \frac{TP}{TP + \alpha FP + (1-\alpha)\, FN} \tag{3}$$

where $TP = \sum_{i=1}^{H}\sum_{j=1}^{W} \hat{y}_{ij} y_{ij}$, $FP = \sum_{i=1}^{H}\sum_{j=1}^{W} (1-\hat{y}_{ij}) y_{ij}$, and $FN = \sum_{i=1}^{H}\sum_{j=1}^{W} \hat{y}_{ij}(1-y_{ij})$ are the true positives, false positives, and false negatives, respectively. Here, α balances false positives and false negatives.

B. Vector Model

The vector model focuses on extracting the vertex set $\mathcal{V}$ and the corresponding vector set $\mathcal{U}$ from the road mask $\boldsymbol{S}_r$. As shown in Fig. 3, the vector model comprises the *road mask encoder*, the *key mask decoder*, the *point decoder*, and the *vector decoder*, which are introduced as follows.

1) *Road Mask Encoder:* It is designed to extract a feature map $\boldsymbol{F}_r$ from the road mask $\boldsymbol{S}_r$ generated by the segmentation model. Note that its structure is streamlined because the road mask $\boldsymbol{S}_r$ has relatively simple semantics. The road mask encoder comprises two downsampling stages with several convolutional layers. Subsequently, the feature map $\boldsymbol{F}_r$ is fed into three subblocks: the key mask decoder, the point decoder, and the vector decoder.

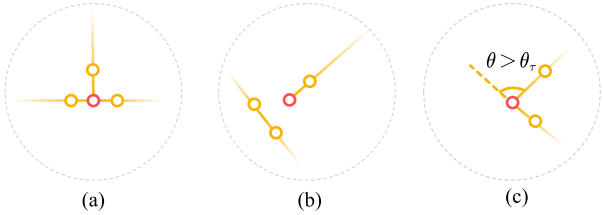


Fig. 4. Key vertex definitions. (a) Intersection vertex with degree greater than 2. (b) Border vertex with degree 1. (c) Elbow vertex with degree 2, where the angle between its two adjacent edges exceeds a threshold θ_τ.

2) *Key Mask Decoder:* As shown in Fig. 2, various graphs with different vertex configurations can represent the same road network. While noncritical vertices may be added or removed, certain vertices—such as intersection points—are critical and must always be included. These are referred to as *key vertices*. The key mask decoder is responsible for generating the key segmentation mask $\boldsymbol{S}_k \in \mathbb{R}^{H\times W}$, which serves as an auxiliary representation for key vertices. In SAM-Road [13], key vertices are defined specifically as vertices with a degree other than 2. In this work, we expand this definition. As illustrated in Fig. 4, key vertices encompass not only those with a degree other than 2 (i.e., intersections and border points) but also "elbow vertices"—vertices with a degree of 2 whose two adjacent edges form an angle greater than the threshold θ_τ. These vertices are crucial for accurate road geometry and topology representation in the graph. The corresponding loss $\mathcal{L}_s^k$ of the key segmentation mask is defined as the BCE loss, consistent with (2).
3) *Point Decoder:* Following SAM-Road [13], the vertices (also referred to as candidate points) of the road graph are sampled via *Non-Maximum Suppression* (NMS) from the road segmentation mask $\boldsymbol{S}_r$ and the key segmentation mask $\boldsymbol{S}_k$. In SAM-Road, sampled vertices are

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.

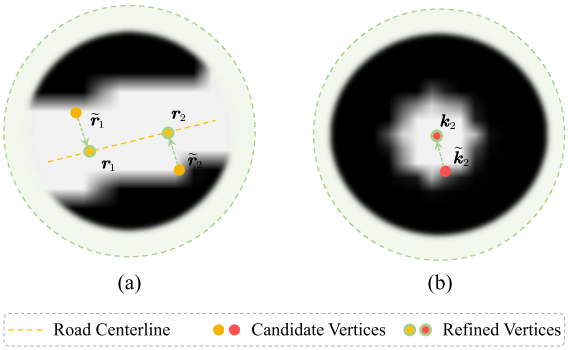


Fig. 5. Vertex refinements. (a) Road vertex refinement. (b) Key vertex refinement.

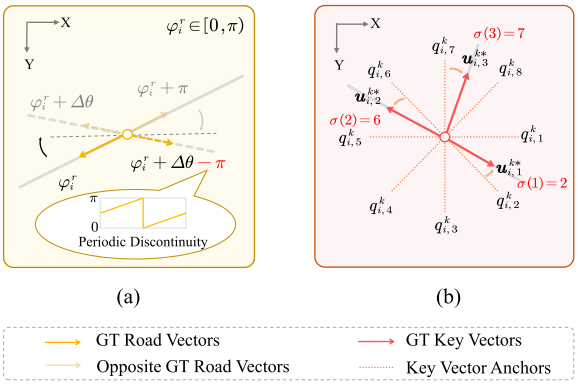


Fig. 6. Road vectors and key vectors. (a) Road vectors. The two road vectors associated with a road vertex are opposite and are periodic with period π. (b) Key vectors and key vector anchors. Key vectors are multidirectional, and their number is not fixed.

directly used as vertices to construct the graph. However, since the positive region in the road mask has a width rather than being a 1-D curve or polyline, the sampled vertices may not align with the center of the road. A similar issue affects key vertices. As illustrated in Fig. 5, the point decoder predicts the offset from the original sampled vertices to the target vertices. In particular, road target vertices are determined as the nearest points on the road centerline to the original vertices. On the contrary, key target vertices are directly assigned as the closest ground-truth key vertices.

Given the sets of road vertices and key vertices defined as $\mathcal{R} \equiv \{\boldsymbol{r}_i \,|\, i = 1, 2, \ldots, N_r\}$ and $\mathcal{K} \equiv \{\boldsymbol{k}_i \,|\, i = 1, 2, \ldots, N_k\}$, the point decoders are implemented as two separate branches for road vertices and key vertices, formulated as follows:

$$\boldsymbol{r}_i \leftarrow \tilde{\boldsymbol{r}}_i + \mathcal{D}_p^r\left(\boldsymbol{F}_r, \tilde{\boldsymbol{r}}_i\right) \tag{4}$$

$$\boldsymbol{k}_i \leftarrow \tilde{\boldsymbol{k}}_i + \mathcal{D}_p^k\left(\boldsymbol{F}_r, \tilde{\boldsymbol{k}}_i\right) \tag{5}$$

where $\tilde{\boldsymbol{r}}_i$ and $\boldsymbol{r}_i$ represent the original and refined coordinates of the ith road vertex, respectively, and $\mathcal{D}_p^r(\cdot,\cdot)$ denotes the road point decoder used to refine the coordinates of road vertices to the road centerline. The key point decoder $\mathcal{D}_p^k(\cdot,\cdot)$ operates analogously. $\tilde{\boldsymbol{k}}_i$ and $\boldsymbol{k}_i$ denote the original and refined coordinates of the ith key vertex, respectively.

The loss for the point decoder, $\mathcal{L}_p$, is defined as follows:

$$\mathcal{L}_p^r = \frac{1}{N_r}\sum_{i=1}^{N_r} \text{SmoothL1}\left(\boldsymbol{r}_i, \boldsymbol{r}_i^*\right) \tag{6}$$

$$\mathcal{L}_p^k = \frac{1}{N_k}\sum_{i=1}^{N_k} \text{SmoothL1}\left(\boldsymbol{k}_i, \boldsymbol{k}_i^*\right) \tag{7}$$

$$\mathcal{L}_p = \mathcal{L}_p^r + \mathcal{L}_p^k \tag{8}$$

where $\boldsymbol{r}_i^*$ and $\boldsymbol{k}_i^*$ represent the target coordinates of the ith road vertex and key vertex, respectively. The smooth L1 loss [41] is applied for the regression part.

4) *Vector Decoder:* The vector decoder is designed to predict directional vectors for each candidate vertex. As illustrated in Fig. 2, these vectors are divided into two categories: "bidirectional" road vectors and "multidirectional" key vectors (hereafter referred to simply as road vectors and key vectors). These are denoted as $\mathcal{U}_r \equiv \{u_i^r \,|\, i = 1, 2, \ldots, N_r\}$ and $\mathcal{U}_k \equiv \{u_i^k \,|\, i = 1, 2, \ldots, N_k\}$, respectively. Each road vertex $\boldsymbol{r}_i$ possesses a corresponding set of road vectors u_i^r, and the same applies to the key vertices. These two types of vectors are described separately below due to their distinct characteristics.

As shown in Fig. 6(a), though one road vector set $u_i^r \equiv \{\boldsymbol{u}_{i,1}^r, \boldsymbol{u}_{i,2}^r\}$ contains two vectors, it has only one degree of freedom, as the two directions are opposite to each other with a phase difference of π. Therefore, we constrain the phase of one road vector to the range $[0, \pi)$ to ensure uniqueness, denoted as $\boldsymbol{u}_i^r$. Nevertheless, as the ground-truth road direction rotates clockwise, the phase of the road vector increases by the corresponding angle (in the image coordinate system). Upon reaching π, it resets to 0 and repeats. Consequently, the phase of the road vector is a periodic function of the ground-truth direction, characterized by periodic discontinuities at 0 and π. This periodic discontinuity is unsuitable for neural network learning because neural networks generally learn continuous mappings more easily. A similar issue arises in rotated object detection [34].

To ensure the continuity of the road vector phase, we double the phase angle and employ a sine-cosine encoding

$$\boldsymbol{\varsigma}_i \equiv \left[\cos\left(2\varphi_i^r\right), \sin\left(2\varphi_i^r\right)\right] \tag{9}$$

where $\boldsymbol{\varsigma}_i$ represents the encoding of the road vector $\boldsymbol{u}_i^r$ for the ith road vertex, and φ_i^r is the phase of $\boldsymbol{u}_i^r$. The phase is doubled because the phase of the road vector is periodic with a period of π, whereas the sine-cosine encoding functions have a periodicity of 2π. The encoding creates a continuous representation that is more conducive to neural network training. Thus, the road vector decoder is formulated as

$$\boldsymbol{\varsigma}_i \leftarrow \mathcal{D}_u^r\left(\boldsymbol{F}_r, \boldsymbol{r}_i\right) \tag{10}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.

and a cosine similarity loss is used to train the road vector decoder

$$\mathcal{L}_u^r = \frac{1}{N_r}\sum_{i=1}^{N_r}\left(1 - \frac{\boldsymbol{\varsigma}_i \cdot \boldsymbol{\varsigma}_i^*}{\|\boldsymbol{\varsigma}_i\|\|\boldsymbol{\varsigma}_i^*\| + \epsilon}\right) \tag{11}$$

where $\boldsymbol{\varsigma}_i^*$ is the corresponding ground-truth encoding of the road vector, with ϵ as a small constant preventing division by zero. During inference, road vectors are obtained by

$$\varphi_i^r = \frac{1}{2}\text{atan2}\left(\boldsymbol{\varsigma}_i^{(y)}, \boldsymbol{\varsigma}_i^{(x)}\right) \mod \pi \tag{12}$$

$$\boldsymbol{u}_i^r = \left[\cos\left(\varphi_i^r\right), \sin\left(\varphi_i^r\right)\right] \tag{13}$$

$$u_i^r \equiv \left\{\boldsymbol{u}_i^r, -\boldsymbol{u}_i^r\right\} \tag{14}$$

where $\boldsymbol{u}_i^r$ is the one recovered road vector for the ith road vertex, and the other road vector is simply its opposite. These two opposite vectors together form the road vector set u_i^r for the ith road vertex.

Distinct from road vectors, key vectors possess multiple directions, as shown in Fig. 6(b). The key vectors of the ith key vertex are denoted by the set $u_i^k \equiv \{\boldsymbol{u}_{i,j}^k \mid j = 1, 2, \ldots, M_i\}$, where M_i represents the number of key vectors for the ith key vertex. Since the number of key vectors is not fixed, we establish a series of key vector anchors characterized by distinct directional phases, denoted as $\boldsymbol{Q} \equiv [q_1; q_2; \ldots; q_{N_a}] \in \mathbb{R}^{N_a}$.

The key vector decoder predicts the phase offsets $\boldsymbol{\phi}_i \equiv [\phi_{i,1}; \phi_{i,2}; \ldots; \phi_{i,N_a}] \in \mathbb{R}^{N_a}$ and confidence scores $\boldsymbol{\rho}_i \equiv [\rho_{i,1}; \rho_{i,2}; \ldots; \rho_{i,N_a}] \in \mathbb{R}^{N_a}$ for the ith key vertex as follows:

$$\left[\boldsymbol{\rho}_i; \boldsymbol{\phi}_i\right] = \mathcal{D}_u^k\left(\boldsymbol{F}_r, \boldsymbol{k}_i\right). \tag{15}$$

Subsequently, the jth key vector of the ith key vertex can be obtained by

$$\boldsymbol{u}_{i,j}^k = \left[\cos\left(q_j + \phi_{i,j}\right), \sin\left(q_j + \phi_{i,j}\right)\right]. \tag{16}$$

It is important to note that some key vector predictions may be invalid and should be treated as negative samples during training. To address this, we employ a one-to-one matching strategy to assign the predicted key vectors to the ground-truth key vectors. In particular, we first calculate the cost matrix between the predicted key vectors and the ground-truth key vectors, and then utilize the Hungarian algorithm [42] to determine the optimal matching. The cost matrix is defined as

$$C_i(m, n) = -\lambda \log\left(\boldsymbol{u}_{i,m}^k \cdot \boldsymbol{u}_{i,n}^{k*} + 1 + \epsilon\right) - \log \rho_{i,m} \tag{17}$$

where $\boldsymbol{u}_{i,n}^{k*}$ is the nth ground-truth key vector of the ith key vertex, and λ balances the cosine-similarity term and the confidence term. Let the optimal matching result be denoted by $\sigma_i(\cdot)$. For instance, $\sigma_i(j)$ indicates that the jth ground-truth key vector is assigned to the $\sigma_i(j)$th predicted key vector, as shown in Fig. 6(b). Consequently, the regression and classification losses for the key vector decoder are formulated as

$$\mathcal{L}_u^k = \frac{1}{\sum_i M_i}\sum_{i=1}^{N_k}\sum_{j=1}^{M_i}\left(1 - \boldsymbol{u}_{i,\sigma_i(j)}^k \cdot \boldsymbol{u}_{i,j}^{k*}\right) \tag{18}$$

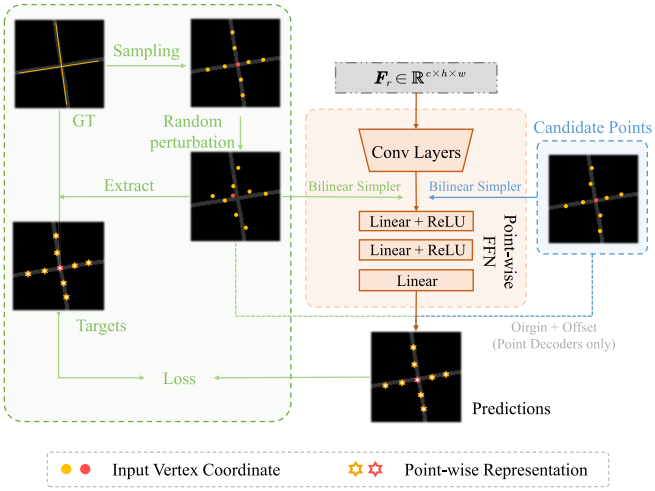


Fig. 7. Typical structure and pipelines of the point and vector decoders. Green arrows and bounding boxes denote the training pipeline, while blue elements indicate the inference pipeline. During training, a teacher-forcing strategy is utilized, feeding vertex points derived from the ground truth as inputs to the decoders.

$$\mathcal{L}_\rho^k = \frac{1}{N_k N_a}\sum_{i=1}^{N_k}\sum_{j=1}^{N_a} -\mathbf{1}_{j\in\xi_i}\log\rho_{i,j} - \mathbf{1}_{j\notin\xi_i}\log\left(1-\rho_{i,j}\right) \tag{19}$$

where ξ_i denotes the set of assigned predicted key vectors for the ith key vertex, defined as $\xi_i \equiv \{\sigma_i(j) | j = 1, 2, \ldots, M_i\}$.

The overall loss for the vector decoders is combined as follows:

$$\mathcal{L}_u = w_u^r\mathcal{L}_u^r + w_u^k\mathcal{L}_u^k + w_\rho^k\mathcal{L}_\rho^k \tag{20}$$

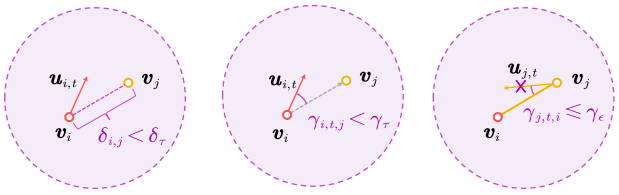
where w_u^r, w_u^k and w_ρ^k are the corresponding weight factors.

C. Training

Although the vector model can be trained independently of the segmentation model, it relies on the road masks generated by the trained segmentation model. In particular, during the training stage of the vector model, the parameters of the segmentation model are frozen. However, it is insufficient to directly use the predicted road mask as input to the vector model, as the candidate points and their corresponding ground-truth labels are not available. To address this issue, we adopt a teacher-forcing strategy during training, feeding vertex points derived from the ground truth as inputs to the decoders, as illustrated in Fig. 7.

In detail, vertex points are sampled from the ground-truth road graph, and noise is added to simulate the sampling errors encountered during inference. These noisy points are then fed into the decoders (e.g., the point decoder or the vector decoder) to predict pointwise representations (e.g., refined point coordinates or directional vectors). The ground-truth labels for these pointwise representations are calculated based on the nearest ground-truth points on the road graph. Although we adopt a teacher-forcing strategy like previous works [13], [18], the vector model learning targets are pointwise representations

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c)

Fig. 8. Geometric quantities for vertex connectivity and road vector removal. (a) Distance between the ith and jth vertices. (b) Angle between the vector of the ith vertex and the direction from the ith vertex to the jth vertex. (c) Road vector removal.

rather than connectivity of multiple vertices, which are more explicit and easier to learn. Therefore, the training-inference gap is significantly reduced compared with previous works.

Fig. 7 also illustrates the typical structure of the four subdecoders, namely the road point decoder, the key point decoder, the road vector decoder, and the key vector decoder. They share the same architecture (without sharing weights) and take points as inputs to predict the corresponding point representations.

The overall training loss of the vector model is formulated as follows:

$$\mathcal{L}_{vec} = \mathcal{L}_s^k + \mathcal{L}_p + \mathcal{L}_u \tag{21}$$

where $\mathcal{L}_s^k$, $\mathcal{L}_p$ and $\mathcal{L}_u$ are the losses of the key mask decoder, the point decoder, and the vector decoder defined in the previous subsections, respectively.

D. Vector-to-Graph

The V2G module is designed to infer the final road graph based on predicted directional vectors. Drawing inspiration from [43], we model connectivity inference as an optimal matching problem, where each directional vector of a vertex is assigned to one candidate vertex.

First, both road and key vertex representations are combined into a unified representation

$$\mathcal{V} \equiv \{\boldsymbol{v}_i \,|\, i = 1, 2, \ldots, N_k, N_k + 1, \ldots, N_k + N_r\} \tag{22}$$

$$\mathcal{U} \equiv \{u_i \,|\, i = 1, 2, \ldots, N_k, N_k + 1, \ldots, N_k + N_r\} \tag{23}$$

$$u_i \equiv \left\{\boldsymbol{u}_{i,j} \,|\, j = 1, 2, \ldots, M_i\right\} \tag{24}$$

where $\mathcal{V} \equiv \mathcal{K} \cup \mathcal{R}$ denotes the unified set of vertices comprising both the key vertices and road vertices, and $\mathcal{U} \equiv \mathcal{U}_k \cup \mathcal{U}_r$ represents the corresponding directional vectors associated with these vertices. Each vertex $\boldsymbol{v}_i$ is associated with a vector set u_i that encompasses one or more directional vector elements.

Two additional geometric quantities are calculated for connectivity inference: 1) the distance $\delta_{i,j}$ between the ith and jth vertices and 2) the angle $\gamma_{i,t,j}$ between the vector $\boldsymbol{u}_{i,t}$ of the ith vertex and the direction from the ith vertex to the jth vertex

$$\delta_{i,j} \equiv \|\boldsymbol{v}_j - \boldsymbol{v}_i\|_2 \tag{25}$$

$$\gamma_{i,t,j} \equiv \arccos\left(\frac{\boldsymbol{v}_j - \boldsymbol{v}_i}{\|\boldsymbol{v}_j - \boldsymbol{v}_i\|_2} \cdot \boldsymbol{u}_{i,t}\right). \tag{26}$$

Two thresholds γ_τ and δ_τ are set for these two quantities, as illustrated in Fig. 8(a) and (b). The candidate connections are first filtered by the thresholds and then finalized through assignment.

Algorithm 1 Vertex Connectivity Algorithm

Require: vertex set $\mathcal{V}$, vector set $\mathcal{U}$, neighbor count M_κ, index range $[i_s, i_e]$, connectivity matrix $\boldsymbol{A}$, thresholds γ_τ and δ_τ
Ensure: Updated connectivity matrix $\boldsymbol{A}$

for $i = i_s$ **to** i_e **do**
  Find the M_κ nearest vertices of $\boldsymbol{v}_i$ excluding itself and denote their indices by $\mathcal{J}_i \equiv [j_1, j_2, \ldots, j_{M_\kappa}]$
  Initialize cost matrix $\boldsymbol{S} \in \mathbb{R}^{M_i \times M_\kappa}$ with all entries set to ∞
  for $t = 1$ **to** M_i **do**
    for $\eta = 1$ **to** M_κ **do**
      $j \leftarrow \mathcal{J}_i[\eta]$
      Calculate $\gamma_{i,t,j}$ and $\delta_{i,j}$
      if $\delta_{i,j} < \delta_\tau$ **and** $\gamma_{i,t,j} < \gamma_\tau$ **then**
        $s_{t,\eta} \leftarrow \delta_{i,j}$
      end if
    end for
  end for
  Obtain the optimal assignment $\pi_i(\cdot)$ from $\boldsymbol{S}$ using the Hungarian algorithm
  for $t = 1$ **to** M_i **do**
    if $s_{t,\pi_i(t)} < \infty$ **then**
      $j \leftarrow \mathcal{J}_i[\pi_i(t)]$
      $a_{i,j} \leftarrow 1$
    end if
  end for
end for
return $\boldsymbol{A}$

The V2G module constructs a connectivity matrix $\boldsymbol{A} = [a_{ij}]_{(N_k+N_r)\times(N_k+N_r)}$, where $a_{i,j} = 1$ indicates an edge connecting the ith and jth vertices, and $a_{i,j} = 0$ otherwise.

The overall V2G algorithm consists of three main steps: key vertex connectivity inference, road vector removal, and road vertex connectivity inference.

Algorithm 1 details the connectivity inference process. In addition to the variables defined above, the algorithm accepts the start and end indices, i_s and i_e, to specify the vertex range for inference. In particular, the key vertex connectivity inference is performed with $i_s = 1$ and $i_e = N_k$, while the road vertex connectivity inference employs $i_s = N_k + 1$ and $i_e = N_k + N_r$. The dimensions of the cost matrix $\mathbf{S} \in \mathbb{R}^{M_i \times M_\kappa}$ are bounded by constants. Therefore, the computational complexity of the connectivity inference is $\mathcal{O}(N_k + N_r)$.

The road vector removal procedure is detailed in Algorithm 2 and visually illustrated in Fig. 8(c). This step is designed to eliminate road vectors that could introduce redundant connections between road vertices. In particular, when a road vertex is linked to a key vertex, certain associated road vectors are removed to prevent it from establishing additional connections to other road vertices.

Finally, the V2G module is executed as follows:

$$\boldsymbol{A} \leftarrow \mathbf{0} \tag{27}$$

$$\boldsymbol{A} \leftarrow f_{vc}(\mathcal{V}, \mathcal{U}, \boldsymbol{A}; i_s = 1, i_e = N_k) \tag{28}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Road Vector Removal Algorithm

Require: Vertex set $\mathcal{V}$, vector set $\mathcal{U}$, connectivity matrix $\boldsymbol{A}$, number of key vertices N_k, number of road vertices N_r, threshold γ_ϵ
Ensure: Updated vector set $\mathcal{U}$

```
for j = N_k + 1 to N_k + N_r do
    for t = M_j down to 1 do
        for i = 1 to N_k do
            Calculate γ_{j,t,i}
            if a_{i,j} = 1 and γ_{j,t,i} < γ_ε then
                Pop u_{j,t} from u_j
                break
            end if
        end for
    end for
end for
return U
```

$$\mathcal{U} \leftarrow f_{vr}(\mathcal{V}, \mathcal{U}, \boldsymbol{A}) \tag{29}$$

$$\boldsymbol{A} \leftarrow f_{vc}(\mathcal{V}, \mathcal{U}, \boldsymbol{A}; i_s = N_k + 1, i_e = N_k + N_r) \tag{30}$$

where f_{vc} denotes the vertex connectivity inference function implemented by Algorithm 1, and f_{vr} represents the road vector removal function implemented by Algorithm 2. Finally, vertices i and j are neighbors in the final road graph if $a_{i,j} = 1$ or $a_{j,i} = 1$.

IV. Experiment

A. Datasets and Metrics

We conduct experiments on three publicly available datasets: City-Scale [10], SpaceNet [30], and Global-Scale [18]. The city-scale dataset comprises 144 training images, nine validation images, and nine test images, each with a resolution of 2048×2048 pixels. The SpaceNet dataset consists of 2040 training images, 127 validation images, and 382 test images, with a resolution of 400×400 pixels per image. The global-scale dataset includes 2375 training images, 339 validation images, 624 in-domain test images, and 130 out-of-domain test images, each with a resolution of 1024×1024 pixels.

We employ TOPO [44] and APLS [30] as our evaluation metrics. TOPO randomly samples target vertices in the ground-truth graph and identifies their corresponding vertices in the predicted graph. Subsequently, it calculates the similarity between the subgraphs rooted at these matched vertices in terms of precision, recall, and the $F1$ score. APLS evaluates topological similarity by computing the shortest paths between pairs of vertices in the ground-truth graph and their corresponding counterparts in the predicted graph. A smaller distance discrepancy indicates a higher degree of similarity.

B. Implementation Details

1) Training: As previously described, the segmentation model and the vector model are trained independently.

For the segmentation model, we set $w_T = 1$, $\alpha = 0.5$. The positive-class weight in the BCE loss is set to 1 on the city-scale and global-scale datasets. Conversely, for the SpaceNet dataset, we configure $w_T = 5$, $\alpha = 0.3$, and set the positive-class weight in the BCE loss to 3. For the vector model, we maintain the following hyperparameter settings across all datasets: $\lambda = 6$, $w_u^r = 1$, $w_u^k = 8$, and $w_\rho^k = 8$, and the elbow threshold θ_τ for the key mask decoder is set to 70° for all datasets. Since intersections and overpasses cannot be distinguished solely based on the road mask, we treat all overpasses as intersections during training.

The segmentation models are trained for 13, 7, and 5 epochs on the city-scale, SpaceNet, and global-scale datasets, respectively. We employ the average weights of the last 3, 2, and 2 epochs [45] for inference. Given the road masks generated by the trained segmentation models, we train the vector models for 6, 4, and 1 epochs on the city-scale, SpaceNet, and global-scale datasets, respectively.

Both the segmentation and vector models use random crops of size 512×512 with a batch size of 16 on the city-scale and global-scale datasets. For the SpaceNet dataset, the patch size is 256×256 with a batch size of 64. All models are optimized using the AdamW optimizer, with an initial learning rate of 10^{-4} and a cosine annealing learning rate scheduler. Data augmentations are consistent with those in SAM-Road [13], including rotations by multiples of 90°, as well as random horizontal and vertical flips.

2) Inference: We employ the same NMS process as SAM-Road [13] to sample initial vertices. The NMS radii ρ_k and ρ_r are set to eight pixels for all datasets. The number of key vector anchors N_a is set to 32, and the number of nearest vertices M_k is set to 8 for all datasets. Regarding connectivity inference, the angle threshold γ_τ is set to 30° for all datasets, while the distance threshold δ_τ is configured to 32 pixels for the city-scale and global-scale datasets, and 20 pixels for the SpaceNet dataset. The angle threshold γ_ϵ for road vector removal is set to 45° across all datasets.

Given the high resolution of remote sensing images, we employ a sliding-window inference strategy to ensure the model processes the entire image, consistent with previous work. The window size is set equal to the training patch size. For the segmentation model, the number of sliding windows is set to 16×16 on the city-scale and global-scale datasets, and 8×8 on the SpaceNet dataset. For the vector model, the number of sliding windows is set to 8×8 across all datasets to balance inference speed and performance. Finally, the V2G module is executed for the entire image to infer the connectivity between vertices and construct the final road graph.

All experiments are conducted on a single RTX 4090 GPU.

C. Comparative Results and Visualization

Tables I and II present the comparative performance of Seg2Vector against other existing methods on the city-scale, SpaceNet, and global-scale test datasets. The results demonstrate that Seg2Vector achieves competitive performance across all three datasets, thereby validating the effectiveness of our approach. Although all methods perform worse on the global-scale out-of-domain test set because of cities unseen during training, our method still achieves the best performance. Our method achieves lower TOPO-P scores

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.

TABLE I

COMPARISON RESULTS OF SEG2VECTOR AGAINST OTHER EXISTING METHODS ON THE CITY-SCALE AND SPACENET TEST DATASETS. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD, AND THE SECOND-BEST RESULTS ARE UNDERLINED. THE UPWARD ARROW INDICATES THAT HIGHER VALUES ARE BETTER

Method	Backbone	City-Scale				SpaceNet			
		TOPO-F1 ↑	TOPO-P ↑	TOPO-R ↑	APLS ↑	TOPO-F1 ↑	TOPO-P ↑	TOPO-R ↑	APLS ↑
Sat2Graph [10]	ResNet-DLA	76.26	80.70	72.28	63.14	80.97	85.93	76.55	64.43
RNGDet [8]	ResNet-101	76.87	85.97	69.87	65.75	81.13	90.91	73.25	65.61
RNGDet++ [9]	ResNet-101	78.44	85.65	72.58	67.76	82.51	91.34	75.24	67.73
TopoRoad [12]	ResNet-101	78.82	79.89	78.10	71.18	83.39	87.88	**81.21**	66.86
PaLiS [11]	ResNet-101	79.08	86.36	73.16	68.12	83.70	90.05	78.19	69.68
GLD-Road [32]	Swin-Small	79.49	83.81	75.77	69.66	**84.72**	92.51	78.15	71.81
SAM-Road [13]	SAM-ViT	77.23	**90.47**	67.69	68.37	80.52	93.03	70.97	71.64
SAM-Road++ [18]	SAM-ViT	80.01	88.39	73.39	68.34	81.57	**93.68**	72.23	73.44
Seg2Vector (Ours)	SAM-ViT	80.72	83.60	78.33	71.05	83.51	90.91	77.22	73.01
Seg2Vector (Ours)	SAM2-ViT-H	**81.66**	83.51	**80.13**	**72.22**	84.52	91.81	78.32	**74.74**

TABLE II

COMPARISON RESULTS OF SEG2VECTOR AGAINST OTHER EXISTING METHODS ON THE GLOBAL-SCALE TEST DATASET. THE TEST DATASET INCLUDES THE IN-DOMAIN AND OUT-OF-DOMAIN SETS

Method	Backbone	Global-Scale (In-Domain)				Global-Scale (Out-of-Domain)			
		TOPO-F1 ↑	TOPO-P ↑	TOPO-R ↑	APLS ↑	TOPO-F1 ↑	TOPO-P ↑	TOPO-R ↑	APLS ↑
Sat2Graph [10]	ResNet-DLA	35.53	90.15	22.13	26.77	30.64	**84.73**	19.75	22.49
RNGDet [8]	ResNet-101	52.59	79.89	40.72	49.43	42.62	68.79	32.60	36.33
RNGDet++ [9]	ResNet-101	55.04	79.02	45.23	52.72	47.34	70.22	35.71	38.08
SAM-Road [13]	SAM-ViT	59.80	**91.93**	45.64	59.08	46.64	84.54	33.81	40.51
SAM-Road++ [18]	SAM-ViT	62.33	88.95	49.27	62.19	48.34	82.21	36.04	43.17
Seg2Vector (Ours)	SAM-ViT	64.80	87.90	52.70	64.72	49.82	77.84	39.12	45.20
Seg2Vector (Ours)	SAM2-ViT-H	**65.31**	88.06	**53.20**	**65.75**	**52.15**	78.06	**41.36**	**48.22**

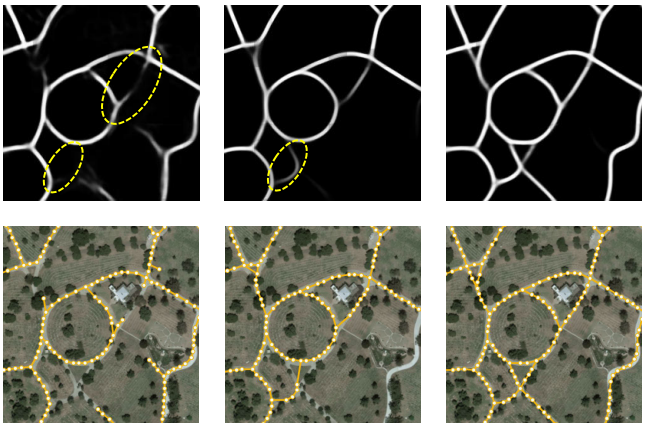


Fig. 9. Visualization of the segmentation results and the corresponding final graph construction.

than SAM-Road and SAM-Road++, but it achieves a better balance between TOPO-P and TOPO-R, leading to higher TOPO-$F1$ scores. In addition to SAM-ViT and SAM2-ViT-H pretrained for segmentation tasks, we conduct experiments on additional backbones for the segmentation model, including ResNet-50 [46], DLA-34 [47], and a self-supervised DINOv3-ConvNext-B [48] backbone. The results are shown in Table III. The segmentation quality of different backbones is evaluated by the mean intersection over union (mIoU).

TABLE III

ABLATION STUDY OF DIFFERENT BACKBONES FOR THE SEGMENTATION MODEL. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD, AND THE SECOND-BEST RESULTS ARE UNDERLINED

Dataset	Backbone	mIoU	TOPO-F1	TOPO-P	TOPO-R	APLS
City-Scale	ResNet-50	54.34	79.97	83.79	76.80	68.92
	DLA-34	54.52	80.00	83.38	77.20	69.93
	DINOv3-ConvNext-B	56.41	80.57	**83.84**	77.85	71.63
	SAM-ViT	57.45	80.72	83.60	78.33	71.05
	SAM2-ViT-H	**58.58**	**81.66**	83.51	**80.13**	**72.22**
SpaceNet	ResNet-50	46.42	82.34	90.53	75.51	71.00
	DLA-34	46.55	82.51	90.37	75.90	71.28
	DINOv3-ConvNext-B	**50.79**	82.97	**93.24**	74.74	73.55
	SAM-ViT	49.39	83.51	90.91	77.22	73.01
	SAM2-ViT-H	50.64	**84.54**	91.82	**78.32**	**74.74**

It can be seen that the final graph extraction performance is positively correlated with the segmentation quality. This indicates that the performance of Seg2Vector can be further enhanced by integrating a more powerful segmentation backbone to generate more precise road masks. Fig. 9 visualizes the segmentation results and the corresponding final graph construction of the different backbones. It can be observed that a higher segmentation quality leads to more accurate and complete road graph predictions.

Fig. 10 shows the road graph predictions of the different methods on the City-Scale test dataset. Fig. 11 visualizes the vector predictions and the final road graph generated by Seg2Vector. The predicted road and key vectors are highly

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.

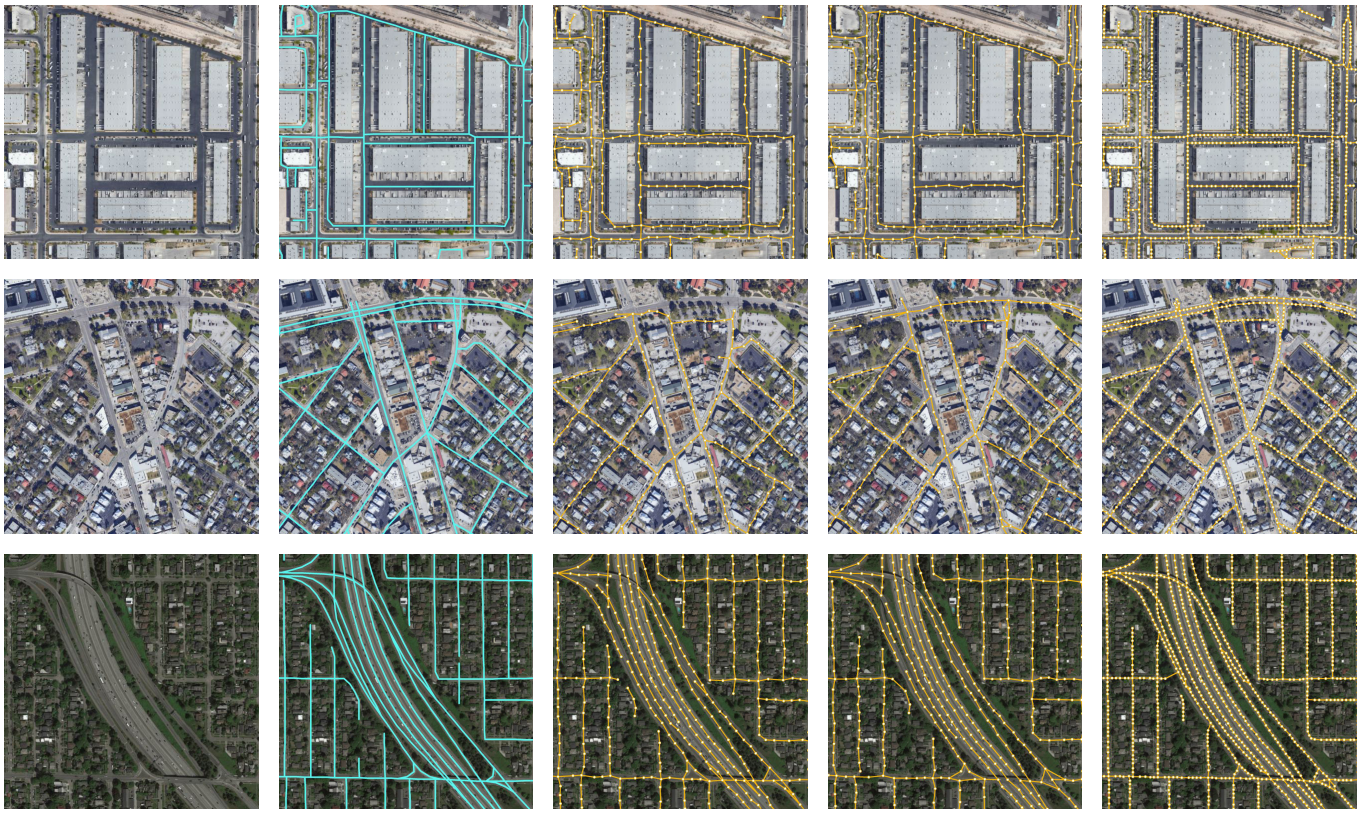


Fig. 10. Visualization of road network graph predictions of different methods on the city-scale test dataset. (a) Original image. (b) Ground truth graph. (c) Prediction of SAM-Road. (d) Prediction of SAM-Road++. (e) Prediction of Seg2Vector.

accurate, particularly for complex structures such as curved roads and intersections.

D. Ablation Study

1) Comparison Between Paradigms: To validate the effectiveness of the proposed vector model and the decoupled paradigm, we conduct an ablation study on the following four paradigms.

1) *Paradigm (a):* SAM-Road (Baseline).
2) *Paradigm (b):* Utilizes segmentation results from SAM-Road but employs a separately trained vector model from our Seg2Vector framework.
3) *Paradigm (c):* A "semi-coupled" design of Seg2Vector, where the vector model takes as input the feature map from the backbone shared with the segmentation model, consistent with the design of SAM-Road.
4) *Paradigm (d):* Our proposed fully decoupled design, where the vector model takes the segmentation mask from a separately trained segmentation model as input.

The results are shown in Table IV. The comparison between paradigms (a) and (b) shows that replacing the vectorization stage with our design substantially improves the topological metrics of SAM-Road. More importantly, comparing paradigms (c) and (d) indicates that the fully decoupled design maintains competitive performance while providing stronger architectural flexibility.

TABLE IV

ABLATION STUDY OF DIFFERENT DECOUPLING PARADIGMS ON THE CITY-SCALE TEST SET. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD, AND THE SECOND-BEST RESULTS ARE UNDERLINED

Strategy	TOPO-F1	TOPO-P	TOPO-R	APLS
(a)	77.13	**90.24**	67.63	68.24
(b)	80.43	83.55	77.83	<u>70.96</u>
(c)	**81.02**	83.11	**79.32**	70.71
(d)	<u>80.72</u>	<u>83.60</u>	<u>78.33</u>	**71.05**

TABLE V

COMBINATIONS OF VECTOR AND SEGMENTATION MODELS ACROSS DIFFERENT DATASETS WITH THE SAM-VIT BACKBONE. THE ROWS AND COLUMNS REPRESENT THE VECTOR AND SEGMENTATION MODELS, RESPECTIVELY. EACH ENTRY IS REPORTED AS TOPO-*F*1/APLS

V \ S	City-Scale	SpaceNet	Global-Scale
City-Scale	80.72 / 71.05	83.23 / 73.04	65.02 / 65.13
SpaceNet	80.27 / 69.73	83.52 / 73.01	64.10 / 63.17
Global-Scale	80.65 / 70.18	83.57 / 73.12	64.80 / 64.72

2) Generalization of the Vector Model: To evaluate the generalization capability of the vector model across different datasets and backbones, we conduct an additional experiment in which the vector model is combined with different segmentation models. As shown in Table V, the vector model

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.

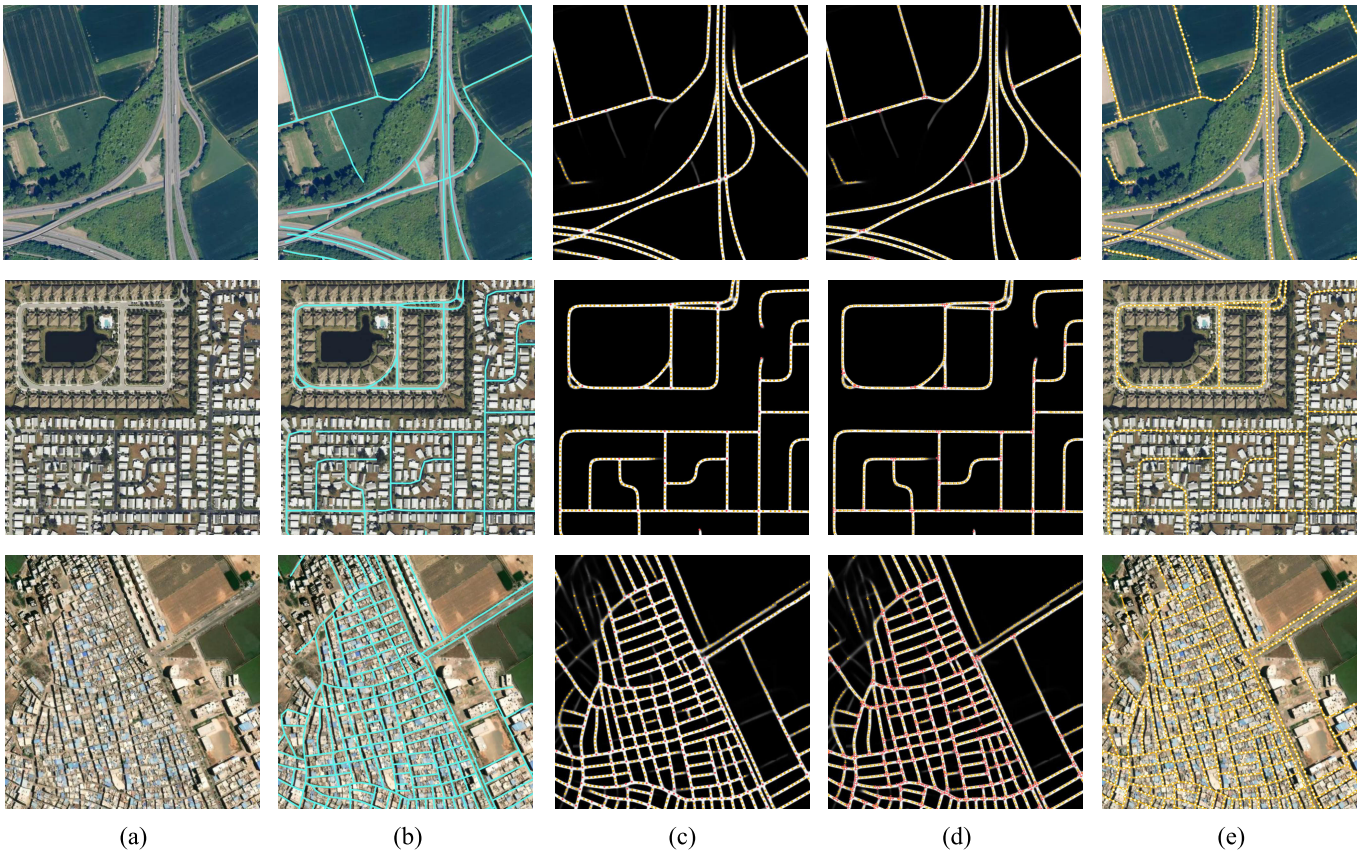


Fig. 11. Visualization of the vector and graph predictions on the global-scale in-domain test dataset. (a) Original image. (b) Ground truth. (c) Refined road and key vertices (yellow and red, respectively). (d) Road vectors (yellow) and key vectors (red). (e) Final road graph.

TABLE VI

COMBINATIONS OF VECTOR AND SEGMENTATION MODELS WITH DIFFERENT BACKBONES ON THE CITY-SCALE TEST DATASET. THE ROWS AND COLUMNS REPRESENT THE VECTOR AND SEGMENTATION MODELS, RESPECTIVELY. EACH ENTRY IS REPORTED AS TOPO-*F*1/APLS

V \ S	ResNet-50	DLA-34	DINOv3	SAM	SAM2
ResNet-50	79.97 / 68.92	79.95 / 69.73	80.53 / 71.54	80.65 / 71.12	81.50 / 71.99
DLA-34	79.89 / 68.87	80.00 / 69.92	80.49 / 71.74	80.68 / 70.90	81.54 / 72.07
DINOv3	79.97 / 68.96	79.98 / 69.99	80.57 / 71.62	80.70 / 71.00	81.63 / 72.37
SAM	79.95 / 68.80	80.02 / 69.72	80.59 / 71.78	80.72 / 71.00	81.63 / 72.18
SAM2	80.01 / 68.77	80.04 / 69.10	80.56 / 71.61	80.75 / 71.08	81.66 / 72.22

TABLE VII

ABLATION STUDY OF KEY VERTICES AND VECTORS

Dataset	Key Vertex	Key Vector	TOPO-F1	TOPO-P	TOPO-R	APLS
City-Scale			78.95	84.70	74.27	68.98
	✓		77.28	85.46	70.92	65.38
	✓	✓	80.72	83.60	78.33	71.05
SpaceNet			83.00	90.99	76.50	71.91
	✓		82.90	91.00	76.12	70.96
	✓	✓	83.51	90.91	77.22	73.01

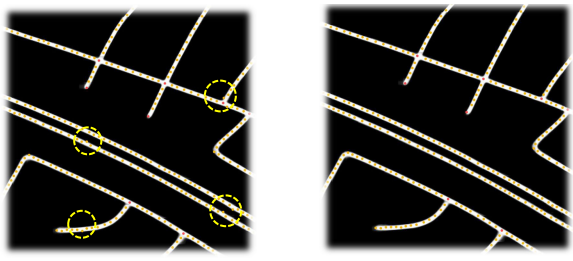


Fig. 12. Effect of the point decoder. (Left) Original vertices sampled by NMS from the road mask and key mask. (Right) Refined vertices.

trained on different source datasets remains competitive with models specifically trained on the target dataset. This indicates that decoupling the segmentation and vector models allows the vector model to generalize well across different datasets. Moreover, as shown in Table VI, the vector model also generalizes well across different segmentation backbones, which further demonstrates the effectiveness of the decoupling paradigm.

3) Key Vertices and Key Vectors: To validate the effectiveness of the key vertices and vectors in the vector model, we conduct an ablation study, as shown in Table VII. We find that key vertices and key vectors contribute to the performance gains. In particular, the key vectors play a significant role in improving the performance. Unlike prior works [11], [12] that model only bidirectional road vectors, our approach models both road and key vectors, leading to improved performance.

4) Point Decoder and Road Vector Removal: We conduct ablation studies to validate the impact of specific architectural

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.

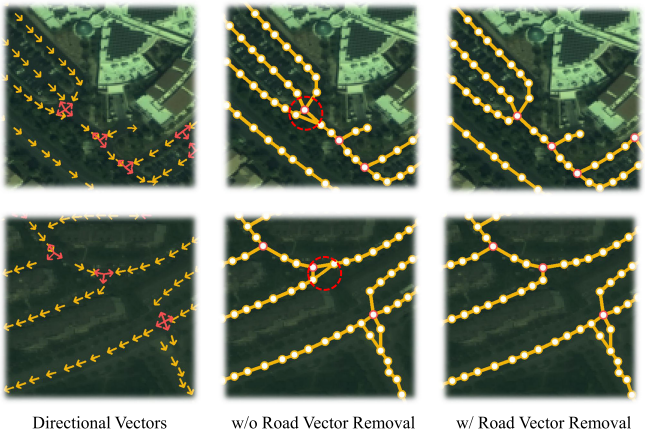


Fig. 13. Effectiveness of road vector removal. Road vector removal helps eliminate redundant connections between road vertices, which can be observed in the red circles.

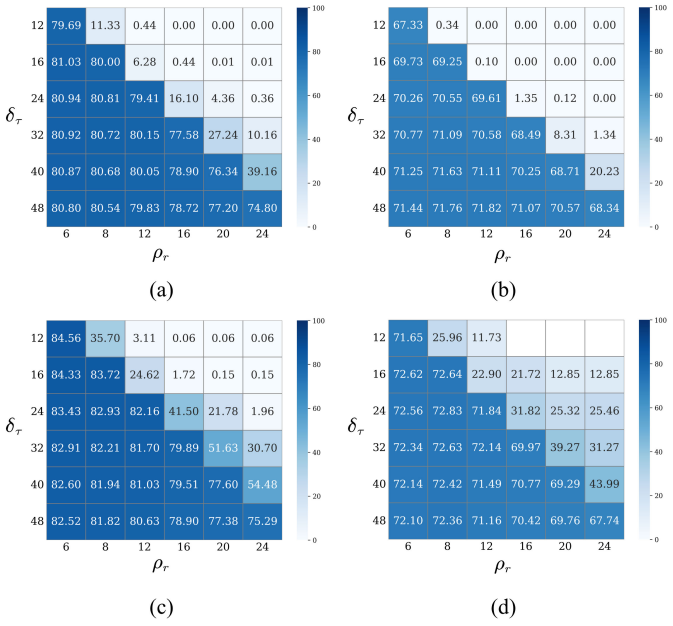


Fig. 14. Ablation study on the relationship between δ_τ and ρ_r. (a) TOPO-$F1$ on the City-Scale test set. (b) APLS on the City-Scale test set. (c) TOPO-$F1$ on the SpaceNet test set. (d) APLS on the SpaceNet test set.

elements on the performance of the point decoder and road vector removal. Though eliminating these individual components had a negligible effect on standard quantitative metrics, the visualization results revealed significant qualitative differences. As illustrated in Fig. 12, the point decoder refines vertex coordinates, yielding more accurate geometries and better alignment with road centerlines. This effectiveness is also corroborated by the results in Fig. 10 when compared with other methods. Furthermore, the road vector removal component effectively eliminates redundant connections between road vertices, which is clearly visible in Fig. 13.

5) Hyperparameter Sensitivity:

1) *Selection of δ_τ and ρ_r:* ρ_r controls the sampling density of road vertices, while δ_τ determines the search radius used in connectivity inference. The experimental results in Fig. 14 in the form of a heatmap indicate that performance drops substantially when $\delta_\tau < 2\rho_r$. This is because the maximum distance between two adjacent vertices can reach up to $2\rho_r$ due to the dynamic distribution of vertices following NMS sampling, as illustrated in Fig. 15. Therefore, to ensure local graph connectivity, it is crucial to set δ_τ greater than $2\rho_r$.

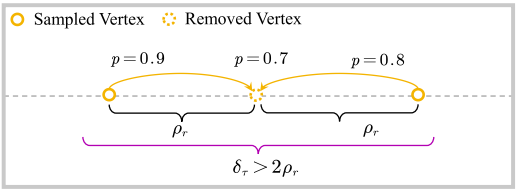


Fig. 15. Illustration of the relationship between δ_τ and ρ_r. Since the vertex distribution is dynamic and the actual distance can reach up to $2\rho_r$, setting δ_τ to be greater than $2\rho_r$ is essential to maintain local graph connectivity.

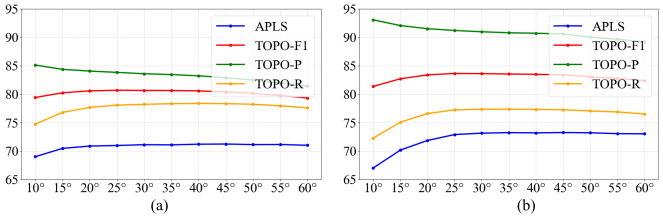


Fig. 16. Ablation study of γ_τ. (a) Results on the City-Scale test set. (b) Results on the SpaceNet test set.

TABLE VIII
INFERENCE TIME AND COMPUTATIONAL COST OF DIFFERENT METHODS ON THE CITY-SCALE AND SPACENET DATASETS

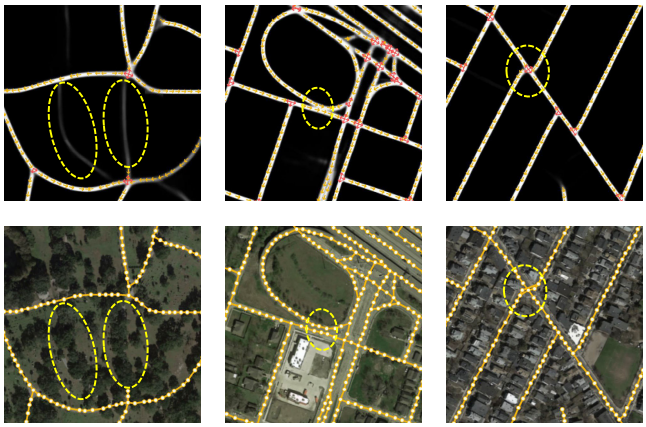
Dataset	Method	Time (min)↓	TFLOPs ↓	Params (M) ↓	Memory (GB) ↓
City-Scale	RNGDet	66.03	72.12	112.26	0.94
	RNGDet++	127.60	192.86	117.43	2.49
	SAM-Road	7.14	58.42	87.82	11.76
	Seg2Vector (SAM)	2.45	61.79	89.18	3.51
	Seg2Vector (SAM2)	2.29	56.34	73.82	3.61
SpaceNet	RNGDet	35.53	2.72	112.26	0.94
	RNGDet++	66.73	7.14	117.43	2.49
	SAM-Road	11.56	17.74	87.21	2.94
	Seg2Vector (SAM)	2.50	5.29	88.57	1.62
	Seg2Vector (SAM2)	2.31	4.75	73.82	1.12

2) *Selection of γ_τ:* Another essential threshold in V2G is γ_τ. The ablation study results, presented in Fig. 16, indicates that performance remains relatively stable when γ_τ ranges from 20° to 50°.

6) Computational Complexity: Although Seg2Vector employs a two-stage decoupled design, it maintains a low computational cost, as shown in Table VIII. Compared with prior methods in our evaluated settings, Seg2Vector shows faster inference and lower memory usage. This efficiency is attributed to vectorized parallel computation, especially in the patch processing stage.

7) Method Boundaries: Fig. 17 illustrates some failure cases of Seg2Vector. Seg2Vector struggles to accurately extract road graphs in some scenarios involving severe occlusion, road tangencies, and complex intersections. Since Seg2Vector relies only on local pointwise features for efficiency, these challenges highlight the boundaries of Seg2Vector and suggest

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c)

Fig. 17. Visualization of some failure cases of Seg2Vector. (Top) Segmentation and directional vectors. (Bottom) Final graph construction results. (a) Segmentation failure. (b) Disconnection between two tangent roads. (c) Redundant connections.

areas for future improvement, such as the incorporation of global context information and more sophisticated feature representations.

V. Conclusion and Future Work

In this article, we propose a novel method, Seg2Vector, for road graph extraction from remote sensing images. By decoupling the segmentation model and the vector model, Seg2Vector effectively leverages the strengths of both approaches to achieve superior performance with significantly reduced computational costs. Extensive experiments on three public datasets demonstrate the effectiveness and efficiency of our method. In future work, we plan to explore more advanced representation learning techniques to further enhance the performance of road graph extraction.

References

[1] R. Lian, W. Wang, N. Mustafa, and L. Huang, "Road extraction methods in high-resolution remote sensing images: A comprehensive review," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 5489–5507, 2020.

[2] M. Büchner, J. Zürn, I.-G. Todoran, A. Valada, and W. Burgard, "Learning and aggregating lane graphs for urban automated driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 13415–13424.

[3] Z. Cai, T. Wang, Q. Mi, X. Su, L. Guo, and Z. Ding, "Dynamic weighted road network based multi-vehicles navigation and evacuation," *ISPRS Int. J. Geo-Information*, vol. 12, no. 3, p. 127, Mar. 2023.

[4] G. Shi, J. Shan, L. Ding, P. Ye, Y. Li, and N. Jiang, "Urban road network expansion and its driving variables: A case study of Nanjing city," *Int. J. Environ. Res. Public Health*, vol. 16, no. 13, p. 2318, Jun. 2019.

[5] Y.-Y. Chiang and C. A. Knoblock, "Extracting road vector data from raster maps," in *Proc. Int. Workshop Graph. Recognit.*, 2009, pp. 93–105.

[6] Z. Li, J. D. Wegner, and A. Lucchi, "Topological map extraction from overhead images," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1715–1724.

[7] Y.-Q. Tan, S.-H. Gao, X.-Y. Li, M.-M. Cheng, and B. Ren, "VecRoad: Point-based iterative graph exploration for road graphs extraction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 8910–8918.

[8] Z. Xu et al., "RNGDet: Road network graph detection by transformer in aerial images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4707612.

[9] Z. Xu, Y. Liu, Y. Sun, M. Liu, and L. Wang, "RNGDet++: Road network graph detection by transformer with instance segmentation and multi-scale features enhancement," *IEEE Robot. Autom. Lett.*, vol. 8, no. 5, pp. 2991–2998, May 2023.

[10] S. He et al., "Sat2Graph: Road graph extraction through graph-tensor encoding," in *Proc. Eur. Conf. Comput. Vis. Cham*, Switzerland: Springer, 2020, pp. 51–67.

[11] J. Xu, B. Xu, G.-S. Xia, L. Dong, and N. Xue, "Patched line segment learning for vector road mapping," *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 6, pp. 6288–6296, Mar. 2024.

[12] Y. Zao, Z. Zou, and Z. Shi, "Topology-guided road graph extraction from remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 4700614.

[13] C. Hetang, H. Xue, C. Le, T. Yue, W. Wang, and Y. He, "Segment anything model for road network graph extraction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2024, pp. 2556–2566.

[14] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.

[15] A. Kirillov et al., "Segment anything," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2023, pp. 4015–4026.

[16] S. Khose, A. Pal, A. Agarwal, Deepanshi, J. Hoffman, and P. Chattopadhyay, "SkyScenes: A synthetic dataset for aerial scene understanding," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 19–35.

[17] D. Wang et al., "HyperSIGMA: Hyperspectral intelligence comprehension foundation model," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 8, pp. 6427–6444, Aug. 2025.

[18] P. Yin et al., "Towards satellite image road graph extraction: A global-scale dataset and a novel method," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 1527–1537.

[19] S. Xie, W. Zheng, Z. Xian, J. Yang, C. Zhang, and M. Wu, "PaRK-detect: Towards efficient multi-task satellite imagery road extraction via patch-wise keypoints detection," 2023, *arXiv:2302.13263*.

[20] C. Heipke, H. Mayer, C. Wiedemann, and O. Jamet, "Evaluation of automatic road extraction," *Int. Arch. Photogramm. Remote Sens.*, vol. 32, no. 3, pp. 151–160, 1997.

[21] J. B. Mena, "State of the art on automatic road extraction for GIS update: A novel classification," *Pattern Recognit. Lett.*, vol. 24, no. 16, pp. 3037–3058, Dec. 2003.

[22] S. Das, T. T. Mirnalinee, and K. Varghese, "Use of salient features for the design of a multistage framework to extract roads from high-resolution multispectral satellite images," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 10, pp. 3906–3931, Oct. 2011.

[23] I. Demir et al., "DeepGlobe 2018: A challenge to parse the Earth through satellite images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2018, pp. 172–181.

[24] V. Mnih, "Machine learning for aerial image labeling: University of Toronto (Canada)," Toronto, ON, Canada, Tech. Rep., 2013.

[25] H. Chen, Z. Li, J. Wu, W. Xiong, and C. Du, "SemiRoadExNet: A semi-supervised network for road extraction from remote sensing imagery via adversarial learning," *ISPRS J. Photogramm. Remote Sens.*, vol. 198, pp. 169–183, Apr. 2023.

[26] Y. Li, R. Zhang, and Y. Wu, "Road network extraction in high-resolution SAR images based CNN features," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2017, pp. 1664–1667.

[27] Y. Lin, D. Xu, N. Wang, Z. Shi, and Q. Chen, "Road extraction from very-high-resolution remote sensing images via a nested SE-deeplab model," *Remote Sens.*, vol. 12, no. 18, p. 2985, Sep. 2020.

[28] Q. Zhu et al., "A global context-aware and batch-independent network for road extraction from VHR satellite imagery," *ISPRS J. Photogramm. Remote Sens.*, vol. 175, pp. 353–365, May 2021.

[29] R. Yang, Y. Zhong, Y. Liu, X. Lu, and L. Zhang, "Occlusion-aware road extraction network for high-resolution remote sensing imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5619316.

[30] A. Van Etten, D. Lindenbaum, and T. M. Bacastow, "SpaceNet: A remote sensing dataset and challenge series," 2018, *arXiv:1807.01232*.

[31] F. Bastani et al., "RoadTracer: Automatic extraction of road networks from aerial images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4720–4728.

[32] L. Deng et al., "GLD-road: A global–local decoding road network extraction model for remote sensing images," *ISPRS J. Photogramm. Remote Sens.*, vol. 228, pp. 741–755, Oct. 2025.

[33] M. Zhang, B. Wang, S. Yang, Q. Liu, and Y. Wang, "KLCRNet: Fast road network extraction via keypoint-driven local connectivity exploration," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 18, pp. 12074–12089, 2025.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.

[34] Y. Yu and F. Da, "Phase-shifting coder: Predicting accurate orientation in oriented object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 13354–13363.
[35] X. Yang, L. Hou, Y. Zhou, W. Wang, and J. Yan, "Dense label encoding for boundary discontinuity free rotation detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 15819–15829.
[36] H. Ling, J. Gao, A. Kar, W. Chen, and S. Fidler, "Fast interactive object annotation with curve-GCN," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 5257–5266.
[37] S. Peng, W. Jiang, H. Pi, X. Li, H. Bao, and X. Zhou, "Deep snake for real-time instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 8533–8542.
[38] N. Xue et al., "Holistically-attracted wireframe parsing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 2788–2797.
[39] N. Xue et al., "Holistically-attracted wireframe parsing: From supervised to self-supervised learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 12, pp. 14727–14744, Dec. 2023.
[40] S. S. M. Salehi, D. Erdogmus, and A. Gholipour, "Tversky loss function for image segmentation using 3D fully convolutional deep networks," in *Proc. 8th Int. Workshop Mach. Learn. Med. Imag.*, 2017, pp. 379–387.
[41] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1440–1448.
[42] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis. Cham*, Switzerland: Springer, 2020, pp. 213–229.
[43] Z. Ge, S. Liu, Z. Li, O. Yoshie, and J. Sun, "OTA: Optimal transport assignment for object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 303–312.
[44] J. Biagioni and J. Eriksson, "Inferring road maps from global positioning system traces: Survey and comparative evaluation," *Transp. Res. Rec., J. Transp. Res. Board*, vol. 2291, no. 1, pp. 61–71, Jan. 2012.
[45] P. Izmailov, D. Podoprikhin, T. Garipov, D. Vetrov, and A. G. Wilson, "Averaging weights leads to wider optima and better generalization," 2018, *arXiv:1803.05407*.
[46] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Proc. Eur. Conf. Comput. Vis. Cham*, Switzerland: Springer, 2016, pp. 630–645.
[47] F. Yu, D. Wang, E. Shelhamer, and T. Darrell, "Deep layer aggregation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2403–2412.
[48] O. Siméoni et al., "DINOv3," 2025, *arXiv:2508.10104*.



Shengqi Wang received the M.S. degree from Xi'an Jiaotong University, Xi'an, China, in 2022, where he is currently pursuing the Ph.D. degree in statistics.

His research interests include computer vision, remote sensing, and representation learning.



Dongyang Zhang received the bachelor's degree from the School of Mathematics and Science, University of Electronic Science and Technology of China, Chengdu, China, in 2021. He is currently pursuing the Ph.D. degree with the School of Mathematics and Statistics, Xi'an Jiaotong University, Xi'an, China.

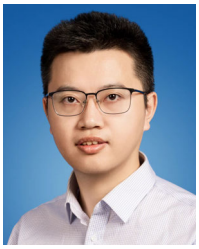
His research interests include continual learning, lifelong learning in computer vision, and natural language processing.



Junmin Liu (Member, IEEE) was born in 1982. He received the Ph.D. degree in mathematics from Xi'an Jiaotong University, Xi'an, China, in 2013.

From 2011 to 2012, he was a Research Assistant with the Department of Geography and Resource Management, The Chinese University of Hong Kong, Hong Kong, China. From 2014 to 2017, he was a Visiting Scholar with the University of Maryland, College Park, MD, USA. He is currently a Professor with the School of Mathematics and Statistics, Xi'an Jiaotong University. He has published more than 60 research papers in international conferences and journals. His main research interests include machine learning and deep learning.

Dr. Liu is a member of CSIAM, the Topic Editor of *Remote Sensing*, the Reviewer Editor of *Frontiers in Computational Neuroscience*, and the reviewer of IEEE TRANSACTIONS ON IMAGE PROCESSING, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, and IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE.



Xiangyong Cao received the B.Sc. and Ph.D. degrees from Xi'an Jiaotong University, Xi'an, China, in 2012 and 2018, respectively.

From 2016 to 2017, he was a Visiting Scholar at Columbia University, New York, NY, USA. He is currently an Associate Professor with the School of Computer Science and Technology, Xi'an Jiaotong University. His research interests include statistical modeling and image processing.



Chengli Tan received the M.S. and Ph.D. degrees in statistics from Xi'an Jiaotong University, Xi'an, China, in 2017 and 2024, respectively.

He is currently an Associate Professor at the School of Mathematics and Statistics, Northwestern Polytechnical University, Xi'an. His research interests include machine learning, Bayesian statistics, and stochastic optimization.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.